

Documentación Técnica Entregable 2

Teoría de Lenguaje de Programación

Santiago Abelardo Salcedo Rodriguez

Dylan Ramirez Blanquicet

Juan Camilo Castano Chavarriaga

6 de Noviembre del 2025

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Motor de Juegos de Ladrillos con Lenguaje Propio (.brik)

1. Introducción

Este documento describe el proceso de diseño e implementación de un motor de juegos basado en ladrillos capaz de ejecutar múltiples videojuegos (actualmente Tetris y Snake), no por hardcoded, sino a partir de un lenguaje de especificación propio (.brik).

La característica distintiva es que el motor no contiene reglas estáticas: el juego que se ejecuta es el que el archivo .brik defina.

2. Pipeline General

1. Entrada: archivo .brik creado por el autor del juego.
2. Lexer y Parser (analyzer.cpp) generan tabla de símbolos.
3. Compilador (compiler.cpp) convierte símbolos a JSON runtime-compatible.
4. Runtime (runtime.cpp) ejecuta el JSON (render, input, lógica).

Módulo 1: Analizador del Lenguaje

Archivo relevante: analyzer.cpp

Este módulo recibe texto fuente .brik y realiza, limpieza de comentarios, lexing (tokenización), Para ello se dividen en distintos tipos de tokens entre ellos podemos encontrar, KEYWORD, STRING, NUMBER, LBRACE, RBRACE etc, se guardan los tokens en un vector que será procesado en la ejecución; Parsing regular mediante regex se verifica que la sintaxis sea correcta a lo largo del archivo y luego se construye la tabla de símbolos.

Sintaxis Válidas.

Comentarios:

// hasta el final de línea

/* ... bloque ... */

String: “ texto”

Identificadores: Admiten todo el Abecedario incluyendo mayúsculas

Símbolos: { } [] () , ; : = . - > :: >= <= == != > < + - * /

Palabras Clave: type, record, ctes, movement, rotation, pieces, clock, scoring, rules, game, using, when, transition, entity, board, tablero, state, line, drop, tickMs, speed, gravity, PLAYING, PAUSE, WON, LOST, APPEARING, ACTIVE, LOCKED, CLEANING, ENTRY, LEFT, RIGHT, DOWN, SPACE, ROTATECW, ROTATECCW, ROTATE180, HOLD, UP

Algunos Bloques Válidos:

type: Declara un nombre de tipo simple (sin campos).

Ejemplo: type NombreTipo;

record: Declara un registro con campos nombre:Tipo.

Ejemplo: record Nombre(x: TipoX, y: TipoY, texto: String);

ctes: Lista de constantes simbólicas (identificadores).

Ejemplo: ctes Direcciones { LEFT RIGHT DOWN UP }

movement: Firma de funciones (solo se registran nombres y “signatures”).

Ejemplo:

```
movement TetrisMoves {  
    moveLeft(Entity) -> State;  
    hardDrop(Board, Piece) -> State;  
}
```

rotation: Análogo a movement, para rotaciones

Forma:

```
rotation SRS {  
    rotCW(Piece) -> Piece;
```

```

    rotCCW(Piece) -> Piece;

}

```

pieces (tetris): Define un conjunto de piezas (I,O,T,S,Z,J,L) con color y shape.

Forma:

```

pieces Tetrominoes using SRS {

  I {

    color = "cyan";

    shape { R0 = [ (0,0), (1,0), (2,0), (3,0) ] }

  }

  T {

    color = "purple";

    shape { R0 = [ (1,0), (0,1), (1,1), (2,1) ] }

  }.

}

```

entity: Entidad con campos tipados (vectoriales permitidos con []).

Forma:

```

entity Player {

    pos: Point;

    trail: Point[];

    speed: Int;

}

```

Módulo 2: Compilador de .brik a JSON

Archivo relevante: compiler.cpp

Este componente toma la tabla de símbolos y genera un JSON con estructura estándar.

El compilador configura, grid_size, eventos ON_START / ON_TICK / ON_KEY_*, puntajes básicos

Para Snake:

- spawn inicial central
- generación comida
- crecimiento y puntaje
- colisiones serían un game_over

Para Tetris:

- spawn pieza aleatoria
- gravedad (DOWN)
- rotaciones
- puntaje al limpiar línea

Un ejemplo en código de esto sería en el archivo compiler.cpp línea 190-220.

```
// ----- build Snake JSON -----
static json build_snake_json(const string& /*source_clean*/){
    json j;
    j["tipo_juego"] = "SNAKE";
    j["config"]["grid_size"] = {20,20}; // tamaño por defecto

    // eventos mínimos para tu runtime.py
    j["events"] = json::object();
    // aparecer jugador en el centro y comida aleatoria
    j["events"]["ON_START"] = json::array({
        { {"accion","SPAWN"}, {"objeto","PLAYER"}, {"params", json::array({ json::array({10,10}) }) } },
        { {"accion","SPAWN"}, {"objeto","FOOD"}, {"params", json::array({"RANDOM"})} }
    });
    // cada tick, mover
    j["events"]["ON_TICK"] = json::array({
        { {"accion","MOVE"}, {"objeto","PLAYER"}, {"params", json::array()} }
    });
    // al comer
    j["events"]["ON_EAT_FOOD"] = json::array({
        { {"accion","INCREASE_SCORE"}, {"objeto",10}, {"params", json::array()} },
        { {"accion","SPAWN"}, {"objeto","FOOD"}, {"params", json::array({"RANDOM"})} },
        { {"accion","GROW"}, {"objeto",nullptr}, {"params", json::array()} }
    });
    // colisiones
    j["events"]["ON_COLLISION_WALL"] = json::array({
        { {"accion","GAME_OVER"}, {"objeto",nullptr}, {"params", json::array()} }
    });
    j["events"]["ON_COLLISION_SELF"] = json::array({
        { {"accion","GAME_OVER"}, {"objeto",nullptr}, {"params", json::array()} }
    });

    return j;
}
```

Módulo 3: Runtime

Archivo relevante: runtime.cpp

Este es el motor real. En nuestro caso utilizamos la librería Curses para mostrar por consola los gráficos necesarios.

Funciones:

Funciones	Detalles
inicializar entorno gráfico	640×480, Curses
ejecutar game loop	para ello recibe inputs que tras ser procesados serán renderizados
Despachar eventos JSON	Ejecuta "SPAWN / MOVE / ROTATE / INCREASE_SCORE / GAME_OVER"
renderizar ladrillos	bloque = "■" en consola
renderizar colores	Ejemplo: snake verde y food roja
renderizar el grid	bloque = "##" en la consola
mapear teclas	flechas y Q (exit)

Un ejemplo de esto puede ser la lectura del teclado por parte del usuario en la linea 167-186

```
//=====
//                                LECTURA DE TECLADO
//=====
void handle_input() {
    int key_code = getch();
    if (key_code == ERR) return;
    if (key_code == 'q' || key_code == 'Q') { game_over = true; return; }

    if (tipo=="SNAKE") {
        if (key_code == KEY_UP) snake_dir(0,-1);
        else if (key_code == KEY_DOWN) snake_dir(0, 1);
        else if (key_code == KEY_LEFT) snake_dir(-1,0);
        else if (key_code == KEY_RIGHT) snake_dir(1, 0);
    } else if (tipo=="TETRIS") {
        if (key_code == KEY_LEFT) t_move("LEFT");
        else if (key_code == KEY_RIGHT) t_move("RIGHT");
        else if (key_code == KEY_DOWN) t_move("DOWN");
        else if (key_code == KEY_UP) t_rotate();
    }
}
```

Donde dependiendo de la tecla presionada por el usuario la serpiente o el tetromino se moverá en la dirección deseada.

Manual de usuario

La implementación del proyecto depende de una librería llamada `curses` la cual permite manipular y crear imágenes o texto en la terminal como si fuera una pantalla 2D.

Al no hacer parte del conjunto estándar de librerías de C++, debe ser descargada de manera externa.

Para facilitar la ejecución del proyecto proponemos múltiples formas de ejecución, las cuales sortean la necesidad de descargar la librería de manera externa pues a la hora de compilar el `runtime.exe` ella fue linkeada estáticamente al archivo. Las formas de ejecución son:

1. La forma más sencilla es desde una carpeta que será entregada aparte, que contiene el archivo `runtime.exe`, `snake.json` y `tetris.json`.

Desde alguna terminal para Windows (WindowsPowershell preferiblemente) abra la ubicación de la carpeta y ejecute alguno de los dos siguientes comandos:

```
.\runtime.exe .\snake.json
```

```
.\runtime.exe .\tetris.json
```

según el juego que desee abrir.

2. También es posible ejecutar el proyecto con los archivos dados desde el github. Tras clonar el repositorio, de nuevo ubicándose con una terminal para windows en la ubicación del proyecto, ejecute alguno de los comandos anteriormente mencionados.

En el caso de que un nuevo `.brik` fuese añadido, el procedimiento para conseguir su `.json` es muy similar al de ejecutar el proyecto.

Inicialmente se debe abrir con alguna terminal de comandos la ubicación del proyecto. Luego se escribe el comando

```
.\compiler.exe .\newGame.brik
```

el cual generará en el código un archivo `.json` nombrado `game.json` por defecto. Este nuevo `.json` puede ser usado luego en conjunción con el `runtime` como fue explicado anteriormente.